```
import pandas as pd
import numpy as np
import warnings
warnings.filterwarnings('ignore')

from sklearn.model_selection import KFold
from sklearn.metrics import mean_absolute_percentage_error
from sklearn.linear_model import Ridge
from sklearn.ensemble import ExtraTreesRegressor
from sklearn.impute import SimpleImputer
from scipy.optimize import minimize

import xgboost as xgb
import lightgbm as lgb
from catboost import CatBoostRegressor

print("All imports successful.")
```

```
---------------------------------------------------------------------------
ModuleNotFoundError                       Traceback (most recent call last)
/tmp/ipykernel_221/1895875461.py in <cell line: 0>()
     13 import xgboost as xgb
     14 import lightgbm as lgb
---> 15 from catboost import CatBoostRegressor
     16
     17 print("All imports successful.")

ModuleNotFoundError: No module named 'catboost'

---------------------------------------------------------------------------
NOTE: If your import is failing due to a missing package, you can
manually install dependencies using either !pip or !apt.

To view examples of installing some common dependencies, click the
"Open Examples" button below.
---------------------------------------------------------------------------
```

OPEN EXAMPLES

```
train_df = pd.read_csv('train.csv')
print(train_df['Target_Variable/Total Income'].median())
```

```
950000.0
```

```
def clean_missing_data(df, target_col='Target_Variable/Total Income'):
    df_clean = df.copy()
    if target_col in df_clean.columns:
        df_clean = df_clean.dropna(subset=[target_col])
    numeric_cols = df_clean.select_dtypes(include=['number']).columns
    categorical_cols = df_clean.select_dtypes(include=['object', 'category']).columns
    for col in numeric_cols:
        if df_clean[col].isnull().any():
            df_clean[col] = df_clean[col].fillna(df_clean[col].median())
    for col in categorical_cols:
        if df_clean[col].isnull().any():
            df_clean[col] = df_clean[col].fillna('Missing')
    return df_clean


def split_temperature_columns(df: pd.DataFrame) -> pd.DataFrame:
    df_processed = df.copy()

    temperature_columns = [
        'K022-Ambient temperature (min & max)',
        'R022-Ambient temperature (min & max)',
        'K021-Ambient temperature (min & max)',
        'R021-Ambient temperature (min & max)',
        'R020-Ambient temperature (min & max)'
    ]

    for col in temperature_columns:
        if col in df_processed.columns:

            split_data = df_processed[col].astype(str).str.split('/', expand=True)

            if split_data.shape[1] == 2:
                df_processed[f'{col.replace(" (min & max)", "_MinTemp")}'] = pd.to_numeric(split_data[0], errors='coerce')
                df_processed[f'{col.replace(" (min & max)", "_MaxTemp")}'] = pd.to_numeric(split_data[1], errors='coerce')
            else:
```

```
                df_processed[f'{col.replace(" (min & max)", "_MinTemp")}'] = np.nan
                df_processed[f'{col.replace(" (min & max)", "_MaxTemp")}'] = np.nan

            df_processed.drop(columns=[col], inplace=True)

    return df_processed


def full_preprocessing_pipeline(train_path, test_path):

    train_df = pd.read_csv(train_path)
    test_df  = pd.read_csv(test_path)

    target_col = 'Target_Variable/Total Income'

    # ---- Temperature column split ----
    train_df = split_temperature_columns(train_df)
    test_df  = split_temperature_columns(test_df)

    # ---- Basic feature cleaning ----
    for df in [train_df, test_df]:

        df['Has_Bureau_History'] = np.where(df['Avg_Disbursement_Amount_Bureau'].isnull(), 0, 1)

        df['Avg_Disbursement_Amount_Bureau'] = df['Avg_Disbursement_Amount_Bureau'].fillna(
            df['Avg_Disbursement_Amount_Bureau'].median()
        )

        df['Location_Missing'] = np.where(df['Location'].isnull(), 1, 0)

        df[['Latitude', 'Longitude']] = df['Location'].str.split(',', expand=True).astype(float)

        df['Latitude'] = df['Latitude'].fillna(df['Latitude'].median())
        df['Longitude'] = df['Longitude'].fillna(df['Longitude'].median())

        df.drop(columns=['Location'], inplace=True)

    # ---- Feature engineering ----
    for df in [train_df, test_df]:

        df['Income_to_Land_Ratio'] = df['Non_Agriculture_Income'] / (df['Total_Land_For_Agriculture'] + 1)

        kharif_irr = [c for c in df.columns if 'Kharif' in c and 'Irrigated' in c]
        rabi_irr   = [c for c in df.columns if 'Rabi' in c and 'Irrigated' in c]

        if kharif_irr and rabi_irr:
            df['Total_Avg_Irrigated_Area'] = df[kharif_irr[0]] + df[rabi_irr[0]]

    # ---- Outlier clipping ----
    outlier_cols = [
        'Avg_Disbursement_Amount_Bureau',
        'Total_Land_For_Agriculture',
        'Non_Agriculture_Income'
    ]

    for col in outlier_cols:
        upper = train_df[col].quantile(0.99)

        train_df[col] = np.clip(train_df[col], None, upper)
        test_df[col]  = np.clip(test_df[col], None, upper)

    # ---- Target encoding ----
    high_card_cols = ['CITY', 'DISTRICT', 'VILLAGE', 'Zipcode', 'K022-Nearest Mandi Name']

    global_mean = np.log1p(train_df[target_col]).mean()

    for col in high_card_cols:

        target_mean = np.log1p(train_df.groupby(col)[target_col].mean())

        train_df[col + '_TE'] = train_df[col].map(target_mean).fillna(global_mean)
        test_df[col + '_TE']  = test_df[col].map(target_mean).fillna(global_mean)

        train_df.drop(columns=[col], inplace=True)
        test_df.drop(columns=[col], inplace=True)

    return train_df, test_df


def prepare_arrays(train_df, test_df, target_col='Target_Variable/Total Income'):
    train_df['Log_Target'] = np.log1p(train_df[target_col])
    y = train_df['Log_Target'].values
```

```
    categorical_cols = train_df.select_dtypes(include=['object']).columns.tolist()
    global_mean = y.mean()
    for col in categorical_cols:
        train_df[col] = train_df[col].fillna('Missing').astype(str)
        test_df[col]  = test_df[col].fillna('Missing').astype(str)
        category_means = train_df.groupby(col)['Log_Target'].mean()
        diff_grade = category_means - global_mean
        train_df[f'{col}_Grade'] = train_df[col].map(diff_grade).fillna(0)
        test_df[f'{col}_Grade']  = test_df[col].map(diff_grade).fillna(0)
        train_df.drop(columns=[col], inplace=True)
        test_df.drop(columns=[col], inplace=True)

    drop_cols = [target_col, 'Log_Target', 'FarmerID', 'Unnamed: 0']
    X      = train_df.drop(columns=[c for c in drop_cols if c in train_df.columns])
    X_test = test_df.drop(columns=[c for c in ['FarmerID', 'Unnamed: 0'] if c in test_df.columns])
    test_ids = test_df['FarmerID'] if 'FarmerID' in test_df.columns else pd.Series(range(len(test_df)))

    imputer = SimpleImputer(strategy='median')
    X      = imputer.fit_transform(X)
    X_test = imputer.transform(X_test)

    return X, y, X_test, test_ids


print("Running preprocessing pipeline...")
train_df_raw, test_df_raw = full_preprocessing_pipeline('train.csv', 'test.csv')
train_df_raw = clean_missing_data(train_df_raw)
test_df_raw  = clean_missing_data(test_df_raw)

X, y, X_test, test_ids = prepare_arrays(train_df_raw, test_df_raw)
print(f"Train shape: {X.shape} | Test shape: {X_test.shape}")
```

```
Running preprocessing pipeline...
Train shape: (10787, 113) | Test shape: (7196, 113)
```

```
N_FOLDS  = 5
SEED     = 42
kf       = KFold(n_splits=N_FOLDS, shuffle=True, random_state=SEED)


def run_oof(model_fn, X, y, X_test, n_folds=N_FOLDS, model_name='Model'):
    """
    Returns:
        oof_preds  : OOF predictions on train set (log scale)
        test_preds : averaged test predictions   (log scale)
        fold_mapes : MAPE per fold (original scale)
    """
    oof_preds   = np.zeros(len(X))
    test_preds  = np.zeros(len(X_test))
    fold_mapes  = []

    for fold, (train_idx, val_idx) in enumerate(kf.split(X, y), 1):
        X_tr, X_val = X[train_idx], X[val_idx]
        y_tr, y_val = y[train_idx], y[val_idx]

        model = model_fn()

        # Route to the correct fit signature per model type
        if isinstance(model, xgb.XGBRegressor):
            model.fit(X_tr, y_tr, eval_set=[(X_val, y_val)], verbose=False)
        elif isinstance(model, lgb.LGBMRegressor):
            # callbacks must be passed here, not in the constructor
            model.fit(X_tr, y_tr,
                      eval_set=[(X_val, y_val)],
                      callbacks=[lgb.early_stopping(50, verbose=False),
                                 lgb.log_evaluation(-1)])
        elif isinstance(model, CatBoostRegressor):
            model.fit(X_tr, y_tr, eval_set=(X_val, y_val), verbose=False)
        else:
            model.fit(X_tr, y_tr)

        val_log_preds          = model.predict(X_val)
        oof_preds[val_idx]     = val_log_preds
        test_preds            += model.predict(X_test) / n_folds

        fold_mape = mean_absolute_percentage_error(np.expm1(y_val), np.expm1(val_log_preds))
        fold_mapes.append(fold_mape)
        print(f"  {model_name} Fold {fold}/{n_folds} — MAPE: {fold_mape*100:.3f}%")

    overall_mape = mean_absolute_percentage_error(np.expm1(y), np.expm1(oof_preds))
    print(f"  ➜ {model_name} OOF MAPE: {overall_mape*100:.3f}%\n")
```

```
    return oof_preds, test_preds, overall_mape
```

```
def make_xgb():
    return xgb.XGBRegressor(
        n_estimators=1000,
        learning_rate=0.05,
        #max_depth=6,
        subsample=0.8,
        colsample_bytree=0.8,
        objective='reg:absoluteerror',
        random_state=SEED,
        n_jobs=-1,
        #early_stopping_rounds=50,
        eval_metric='mae'
    )


def make_lgbm():
    # LightGBM — faster and often slightly better than XGB on tabular data
    # NOTE: callbacks must go in .fit(), NOT in the constructor
    return lgb.LGBMRegressor(
        n_estimators=1500,
        learning_rate=0.03,
        max_depth=7,
        num_leaves=63,
        subsample=0.8,
        colsample_bytree=0.8,
        reg_alpha=0.1,
        reg_lambda=0.1,
        min_child_samples=20,
        objective='regression_l1',   # MAE loss — consistent with XGB reg:absoluteerror
        random_state=SEED,
        n_jobs=-1,
        verbose=-1
    )


def make_catboost():
    return CatBoostRegressor(
        iterations=2000,
        learning_rate=0.05,
        depth=7,
        l2_leaf_reg=3,
        loss_function='MAE',
        eval_metric='MAPE',
        random_seed=SEED,
        early_stopping_rounds=50,
        verbose=0
    )


print("Model factories defined.")
```

```
Model factories defined.
```

```
print("=" * 55)
print("TRAINING XGBoost")
print("=" * 55)
xgb_oof,  xgb_test,  xgb_mape  = run_oof(make_xgb,        X, y, X_test, model_name='XGBoost')

print("=" * 55)
print("TRAINING LightGBM")
print("=" * 55)
lgb_oof,  lgb_test,  lgb_mape  = run_oof(make_lgbm,       X, y, X_test, model_name='LightGBM')

print("=" * 55)
print("TRAINING CatBoost")
print("=" * 55)
cat_oof,  cat_test,  cat_mape  = run_oof(make_catboost,   X, y, X_test, model_name='CatBoost')

print("\n" + "=" * 55)
print("BASE MODEL SUMMARY")
print("=" * 55)
print(f"  XGBoost    OOF MAPE: {xgb_mape*100:.3f}%")
print(f"  LightGBM   OOF MAPE: {lgb_mape*100:.3f}%")
print(f"  CatBoost   OOF MAPE: {cat_mape*100:.3f}%")
```

```
============================================================
TRAINING XGBoost
============================================================
  XGBoost Fold 1/5 — MAPE: 19.069%
  XGBoost Fold 2/5 — MAPE: 17.789%
  XGBoost Fold 3/5 — MAPE: 17.485%
---------------------------------------------------------------------------
KeyboardInterrupt                         Traceback (most recent call last)
/tmp/ipykernel_223/4069433455.py in <cell line: 0>()
      2 print("TRAINING XGBoost")
      3 print("=" * 55)
----> 4 xgb_oof, xgb_test, xgb_mape = run_oof(make_xgb,X, y, X_test, model_name='XGBoost')
      5
      6 print("=" * 55)

 ⇕ 5 frames
/usr/local/lib/python3.12/dist-packages/xgboost/core.py in update(self, dtrain, iteration, fobj)
   2389         if fobj is None:
   2390             _check_call(
-> 2391                 _LIB.XGBoosterUpdateOneIter(
   2392                     self.handle, ctypes.c_int(iteration), dtrain.handle
   2393                 )

KeyboardInterrupt:
```

```
# Stack OOF preds: shape (n_train, n_models) — in LOG space
oof_matrix  = np.column_stack([xgb_oof, lgb_oof, cat_oof, et_oof])
test_matrix = np.column_stack([xgb_test, lgb_test, cat_test, et_test])
model_names = ['XGBoost', 'LightGBM', 'CatBoost', 'ExtraTrees']

y_true_orig = np.expm1(y)   # original scale for MAPE


def mape_from_weights(weights, oof_matrix=oof_matrix, y_true=y_true_orig):
    """Compute MAPE given a weight vector over the OOF columns."""
    weights = np.array(weights)
    blended_log = oof_matrix @ weights           # weighted sum in log space
    blended     = np.expm1(blended_log)          # back to original scale
    return mean_absolute_percentage_error(y_true, blended)


n_models = oof_matrix.shape[1]
init_weights = np.ones(n_models) / n_models     # start at equal weights

constraints = ({'type': 'eq',   'fun': lambda w: w.sum() - 1.0})   # weights sum to 1
bounds      = [(0.0, 1.0)] * n_models                               # non-negative

result = minimize(
    mape_from_weights,
    init_weights,
    method='SLSQP',
    bounds=bounds,
    constraints=constraints,
    options={'maxiter': 1000, 'ftol': 1e-9}
)

optimal_weights = result.x
blend_mape      = result.fun

print("Optimized weights:")
for name, w in zip(model_names, optimal_weights):
    print(f"  {name:<12}: {w:.4f}  ({w*100:.1f}%)")
print(f"\nBlended OOF MAPE (weighted avg): {blend_mape*100:.3f}%")

# Generate test predictions with optimized weights
blend_test_log = test_matrix @ optimal_weights
blend_test     = np.expm1(blend_test_log)

submission_blend = pd.DataFrame({'FarmerID': test_ids, 'Target_Variable/Total Income': blend_test})
submission_blend.to_csv('submission_weighted_blend.csv', index=False)
print("\nSaved: submission_weighted_blend.csv")
```

Double-click (or enter) to edit